[redacted]

[redacted]

Faculty of [redacted]

A PROJECT REPORT
ON
SWASTHYA TRACK

Submitted to

[redacted]

[redacted]

In partial fulfillment of the requirements for [redacted]

Submitted by

Unish Rajak

[redacted]

Under the supervision of

[redacted]

February, 2026

[redacted]

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project prepared under my supervision by **Unish Rajak** entitled **"SwasthyaTrack"** in partial fulfillment of the requirements for the degree of Bachelor of Computer Application is recommended for the final evaluation.

……..

[redacted]

[redacted]

Faculty of [redacted]

LETTER OF APPROVAL

This is to certify that this project prepared by **Unish Rajak** entitled **"SwasthyaTrack"** in partial fulfillment of the requirements for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

……. [redacted] **SUPERVISOR** [redacted]	…….. [redacted] [redacted]**COORDINATOR** [redacted]
…….. **Internal Examiner**	…….. **External Examiner**

ABSTRACT

SwasthyaTrack is a web-based hospital management system developed using the LAMP stack (Linux, Apache, MySQL, PHP). The system aims to make hospital operations easier by combining different tasks such as patient registration, appointment booking, bed management, and medical record storage into one platform. It allows patients to create accounts, book and manage appointments, reserve beds, and upload their medical reports. Doctors can view appointments, write prescriptions, and access patient reports, while administrators and nurses can manage users, appointments, and hospital resources through a dashboard.

The system focuses on improving communication between patients and hospital staff and reducing paperwork by digitizing common hospital activities. Passwords are stored securely using hashing, and all files are handled safely to protect user data. SwasthyaTrack is developed in a Linux-based environment with docker for consistent deployment and easier maintenance. Overall, the system provides a simple, reliable, and secure way to manage hospital services efficiently, improving both patient experience and hospital workflow.

Keywords: ***Hospital Management, Appointment Booking, Medical Records, LAMP Stack, Electronic Medical Records***

ACKNOWLEDGEMENT

We would like to express our deepest appreciation to all those who provided us with the possibility to complete this report. We also thank our [redacted] for supporting and managing us in our project. A special gratitude from me to my project Supervisor, [redacted] whose contribution stimulating suggestions and encouragement, helped us to coordinate our project especially in drafting this report. We would also like to acknowledge the vital role played by the staff of [redacted], who gave us the approval to use all required equipment and the necessary materials to complete the task.

Furthermore, we would like to give a special thanks to all the associated people from [redacted] department and other departments, including our seniors who have guided us along the way and made the project successful. We would also like to share our sincere gratitude to our other supervisors and the panel, whose critics and advice during our presentations have enhanced our knowledge and truly urged our project to reach its optimum potential. And last but not the least we would like to thank every single one of our colleagues for their motivation during the development of this project.

With respect,

Unish Rajak

[redacted]

TABLE OF CONTENTS

SUPERVISOR'S RECOMMENDATION i

LETTER OF APPROVAL ii

ABSTRACT iii

ACKNOWLEDGEMENT iv

TABLE OF CONTENTS v

LIST OF ABBREVIATIONS vii

LIST OF FIGURES viii

LIST OF TABLES ix

CHAPTER 1: INTRODUCTION 1

1.1 Introduction 1

1.2 Problem Statement 2

1.3 Objectives 4

1.4 Scope and Limitation 4

1.4.1 Scope: 4

1.4.2 Limitations 5

1.5 Report Organization 5

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW 7

2.1 Background Study 7

2.1.1 Study of some other existing systems 8

2.2 Literature review 10

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN 12

3.1 System Analysis 12

3.1.1 Requirement Analysis 12

3.1.2 Feasibility Analysis 14

3.1.3 Data Modelling 17

3.1.4 Process Modeling 18

3.2 System Design 19
3.2.1 Architectural Design 19
3.2.2 Database Schema Design 21
3.2.3 Interface Design 22
3.2.4 Physical DFD 25
CHAPTER 4: IMPLEMENTATION AND TESTING 26
4.1 Implementation 26
4.1.1 Tools Used 26
4.1.2 Implementation Details of Modules 27
4.2 Testing 30
4.2.1 Test Cases for Unit Testing 30
4.2.2 Test Cases for System Testing 58
CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS 63
5.1 Lesson Learnt / Outcome 63
5.2 Conclusion 63
5.3 Future Recommendations 64
APPENDICES
REFERENCES
STUDENT PROGRESS RECORD SHEET

LIST OF ABBREVIATIONS

CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DB	Database
DFD	Data Flow Diagram
EMR	Electronic Medical Record
ERD	Entity Relationship Diagram
GDPR	General Data Protection Regulation
HIPAA	Health Insurance Portability and Accountability Act
HMS	Hospital Management System
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JS	JavaScript
LAMP	Linux, Apache, MySQL, PHP
PHP	Hypertext Preprocessor
RBAC	Role-Based Access Control
SQL	Structured Query Language
UI/UX	User Interface / User Experience
WSL2	Windows Subsystem for Linux 2

LIST OF FIGURES

Figure 3.1 Use Case Diagram of SwasthyaTrack 13

Figure 3.2 Schedule Timeline 16

Figure 3.3 Entity Relation Diagram of SwasthyaTrack 17

Figure 3.4 DFD Level-0 of SwasthyaTrack 18

Figure 3.5 DFD Level-1 of SwasthyaTrack 19

Figure 3.6 Architectural Diagram 20

Figure 3.7 Database Schema of SwasthyaTrack 21

Figure 3.8 Landing Page 22

Figure 3.9 Login Page 22

Figure 3.10 Signup Page 23

Figure 3.11 User's Dashboard 23

Figure 3.12 Book Bed Page 24

Figure 3.13 Admin & Nurse Dashboard 24

Figure 3.14 Physical DFD of SwasthyaTrack 25

LIST OF TABLES

Table 4.1 Tools & Technology 26

Table 4.2 Signup Unit Test 30

Table 4.3 Login Unit Test 35

Table 4.4 Book Appointment Unit Test 37

Table 4.5 Book Bed Unit Test 41

Table 4.6 Edit Appointment Unit Test 46

Table 4.7 Edit Bed Reservations Unit Test 52

Table 4.8 Upload Your Medical Report Unit Test 57

Table 4.9 System Testing 58

CHAPTER 1: INTRODUCTION

1.1 INTRODUCTION

SwasthyaTrack is a hospital management system designed to simplify and modernize hospital operations. The main aim of this project is to replace traditional manual processes with an automated digital system that allows hospitals to manage their day-to-day activities more efficiently. It provides an integrated platform where patients, doctors, nurses, and administrators can perform their respective tasks through a single web application.

It offers a centralized system that stores all information in a secure and easily accessible database. Patients can book appointments online, view their prescriptions, check bed availability, and upload their medical reports directly to the system. Doctors can view their upcoming appointments, access patient records, and issue prescriptions digitally. Nurses can upload medical reports and help manage patient appointments, while administrators can oversee all user accounts, bed allocations, and hospital data from a single dashboard. This structured division of roles ensures that the system functions smoothly and maintains data privacy for every user.

The project is developed using LAMP stack (Linux, Apache, MySQL, PHP) which provides a stable and widely used web development environment. The Linux environment required for the project is provided through WSL2 (Windows Subsystem for Linux 2), a Windows feature that enables the execution of Linux-based systems directly within a Windows operating system. This allows developers to run and test Linux commands, manage services like Apache and MySQL, and configure Docker containers without switching to a separate machine. It ensures a consistent and reliable setup during development and deployment.

The front end of SwasthyaTrack is designed using HTML, CSS, and JavaScript to create a clean and user-friendly interface. The backend is developed using PHP to handle server-side logic, and MySQL serves as the main database for storing all user and hospital information. Docker is used for containerization, which simplifies deployment and ensures that the system runs consistently across different computers and servers. All passwords are stored securely using hashing techniques to protect user data and maintain confidentiality.

In terms of functionality, SwasthyaTrack provides several modules such as user management, appointment scheduling, bed management, report handling, and prescription management. Each module works together to streamline hospital operations. For example, when a patient books an appointment, the doctor's schedule automatically updates, and both parties receive confirmation. Similarly, when a nurse uploads a report, it becomes visible to the doctor during the next consultation. This level of integration reduces communication gaps and ensures that the right information reaches the right people at the right time.

The system also helps hospital administrators by providing a real-time overview of the entire operation through a dashboard. This assists management in decision-making and resource allocation. By reducing paperwork, automating scheduling, and maintaining a complete digital record of hospital activities, SwasthyaTrack increases overall efficiency, accuracy, and patient satisfaction.

SwasthyaTrack fulfills the need of efficient systems that can handle large amounts of data securely and provide instant access to medical information by offering a reliable and secure digital solution that supports doctors, patients, and administrative staff in their daily activities. It promotes better communication, reduces manual errors, and ensures that every record is maintained systematically.

SwasthyaTrack is a step toward smarter healthcare management. It combines modern technology with practical usability to create a system that is not only easy to use but also capable of improving hospital workflow and patient care. The successful implementation of this system demonstrates how digital innovation can enhance healthcare delivery, support evidence-based decision-making, and build a foundation for future integration with larger medical systems.

1.2 PROBLEM STATEMENT

No projects or applications are 100% productive, to overcome the flaw in existing application new projects and innovations are carried out. Following are the problems that hospitals are facing:

- **Manual Record-Keeping:** Most hospital records are still maintained on paper, which makes data handling time-consuming and prone to human error. Important

files can be misplaced or damaged, making it difficult to retrieve patient information when needed.

- **Lack of Centralized Data Storage:** Patient information is often scattered across different departments. Without a centralized database, doctors, nurses, and administrators cannot easily access complete medical histories or previous reports.

- **Inefficient Appointment Scheduling:** Appointments are typically managed through manual entries or phone calls. This leads to double bookings, missed consultations, and long waiting times for patients.

- **Unorganized Bed Management:** Hospitals without a proper bed management system struggle to keep accurate track of available and occupied beds. This creates confusion during patient admissions and emergencies.

- **Weak Data Security:** Paper-based and unprotected digital records expose sensitive patient information to risks such as data loss, theft, or unauthorized access. There is a lack of proper encryption and authentication mechanisms.

- **Poor Communication Among Staff:** Doctors, nurses, and administrative personnel often communicate verbally or through handwritten notes, leading to delays and miscommunication about patient updates and treatment progress.

- **Inconsistent Patient Status Updates:** Patient status, including progress notes, test results, and treatment responses, is not consistently updated or communicated across departments. This lack of reliable, real-time information prevents healthcare professionals from making well-informed decisions. It also disrupts collaboration among team members and increases the likelihood of errors in treatment planning.

- **Increased Workload and Human Error:** Repetitive data entry and manual coordination increase the workload for staff, reduce productivity, and lead to frequent mistakes in records or scheduling.

These problems highlight the need for an automated and centralized hospital management system. SwasthyaTrack is designed to solve these challenges by digitizing hospital operations, improving coordination among departments, enhancing data security, and ensuring faster, more accurate healthcare service delivery.

1.3 OBJECTIVES

In the case for developing SwasthyaTrack, following are the objective that this project aims to achieve:

- To develop a LAMP-based web application for hospital management, integrating patient, clinical, and administrative functions.
- To provide an online appointment booking system that allows patients to schedule, modify, or cancel appointments with doctors conveniently without manual interaction.
- To build a unified system that gives medical staff instant access to the appointments alongside with patient's full medical history.

1.4 SCOPE AND LIMITATION

1.4.1 Scope:

The scope of the SwasthyaTrack system is limited to the following functional and operational boundaries:

- The system provides a web-based hospital management platform for digitizing core hospital operations.
- It supports user role segregation, including patients, doctors, nurses, and administrators, with role-based access control.
- Patients can register, log in, book appointments, reserve beds, upload medical reports, and view prescriptions through the system.
- Doctors can view scheduled appointments, access patient medical records, and generate digital prescriptions.
- Nurses can manage appointments and upload medical reports for patients as part of clinical support activities.
- Administrators can manage users, appointments, bed availability, and overall system data through an administrative dashboard.
- The system maintains a centralized database for storing patient information, appointments, prescriptions, and reports securely.

- SwasthyaTrack is developed using the LAMP stack (Linux, Apache, MySQL, PHP) and deployed using Docker within a WSL2 environment.
- The application is intended for small to medium-sized hospitals or clinics seeking an affordable and basic digital management solution.
- The scope of the project is confined to academic implementation and demonstration, rather than enterprise-level healthcare deployment.

1.4.2 Limitations

- Supports only core hospital management functions; advanced modules such as real-time billing integration, inventory control, and insurance processing are currently out of scope.
- Operates as a web-based system, requiring a stable and continuous internet connection for optimal performance.
- Developed primarily as an academic prototype, with no enterprise-scale deployment or high-volume performance testing conducted to date.
- Lacks mobile application integration, limiting accessibility on handheld devices.
- Does not currently offer advanced analytics or decision-support features.
- Multi-hospital or multi-branch management is not supported in the present version, restricting scalability for larger healthcare networks.

1.5 REPORT ORGANIZATION

This report is organized according to the standard structure prescribed for the BCA Project Report. It includes all essential sections from the cover page to the appendices, ensuring a logical and systematic flow of information.

The Main Report is divided into five chapters:

- **Chapter 1: Introduction:** Presents an overview of the project, including the introduction, problem statement, objectives, scope, limitations, and report organization.

- **Chapter 2: Background Study and Literature Review**: Describes the theoretical background, related concepts, and previous works or similar systems studied during research.
- **Chapter 3: System Analysis and Design**: Includes requirement analysis, feasibility study, data and process modeling, and system design such as architecture, database schema, and interface design.
- **Chapter 4: Implementation and Testing:** Explains the implementation details, tools used, developed modules, and test cases used for validating the system's performance.
- **Chapter 5: Conclusion and Future Recommendations**: Summarizes the project outcomes, conclusions, and possible future improvements.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 BACKGROUND STUDY

Hospital Management Systems (HMS) have become a critical part of modern healthcare. They integrate administrative, clinical, and operational activities into a single digital platform, helping hospitals improve efficiency, accuracy, and overall patient care. Systems such as SwasthyaTrack reflect the growing need for reliable and intelligent digital solutions in healthcare environments.

A core element of any HMS is the Electronic Medical Record (EMR). EMRs replace paper-based records with digital data that can be accessed and updated in real time. They store essential information such as patient details, diagnoses, prescriptions, and lab results. By making patient data readily available, EMRs support better clinical decisions, reduce documentation errors, and eliminate redundant procedures. Global health authorities emphasize that EMRs enhance both safety and continuity of care. [1]

Online appointment scheduling is another important feature. Traditional booking methods often lead to delays, missed appointments, and administrative inefficiencies. Digital scheduling systems allow patients to conveniently book or manage appointments through a web interface while enabling hospitals to coordinate doctor availability and resources more effectively. Studies indicate that such systems improve patient satisfaction and workflow management. [2]

Efficient bed and patient flow management is also essential. Without centralized tracking, hospitals may experience overcrowding or delays in admissions and transfers. Digital bed management modules provide real-time visibility into occupancy and availability, helping staff manage admissions, discharges, and transfers smoothly. Research shows that these systems significantly reduce operational bottlenecks. [3]

Data security is a fundamental requirement for healthcare systems. SwasthyaTrack implements Role-Based Access Control (RBAC), where access permissions are assigned based on user roles. This ensures that users only access information relevant to their responsibilities, supporting privacy, accountability, and the principle of least privilege. RBAC is widely recognized as an effective security model in healthcare systems. [4]

Interoperability is another key consideration. Hospitals rely on multiple systems and diagnostic tools that must communicate effectively. Standards such as HL7 and FHIR enable structured data exchange, ensuring that patient information remains consistent and complete across platforms. Poor interoperability can lead to fragmented care, while integrated systems improve clinical coordination. [5]

Compliance with privacy and data protection regulations is equally important. Frameworks such as HIPAA and GDPR define how medical data should be handled, stored, and secured. SwasthyaTrack incorporates mechanisms such as encrypted connections, hashed credentials, and audit logs to align with these requirements, thereby enhancing trust and legal compliance. [6] [7].

Finally, usability plays a significant role in system success. HMS platforms serve diverse users with varying needs, including medical staff and patients. Well-designed, user-friendly interfaces reduce errors, improve productivity, and encourage adoption. Human-centered design principles ensure that users can interact with the system efficiently and intuitively. [8].

The foundational concepts behind SwasthyaTrack, EMRs, online scheduling, bed management, RBAC, interoperability, security compliance, and usability are essential to the success of any hospital management solution. By integrating these components into a modular, scalable web application, SwasthyaTrack aligns with global best practices in healthcare informatics and is well-positioned to address the operational and clinical needs of contemporary hospitals.

2.1.1 Study of some other existing systems

Hamro Doctor:

Hamro Doctor is a widely used digital health application that provides online medical consultations and health-related information. It allows users to connect with doctors at lower costs compared to physical visits. The platform includes features such as doctor listings by specialization, details of health checkup packages offered by hospitals, ambulance contacts, medical blogs, and live chat options for health queries. [9]

The application organizes services into categories like Skin, Heart, ENT, Thyroid, and Women's Health, which simplifies navigation. Additional features include health news updates, blood donation information, and the ability to upload medical reports. These

functions make healthcare access more convenient and reduce the need for in-person hospital visits. [9]

Cura Health:

Cura Health was built to make sure that people can seek medical help without losing out on valuable time, digitally via this app. The app is designed to bridge the gap between patients and medical professionals. Unlike other health and medical apps, users cannot enter the home page without registering or signing in, and providing all their vital statistics (height, weight, blood type, disease history, etc.), as well as an emergency contact number, a thoughtful touch that physical forms at hospitals make mandatory but other apps seem to have skipped. [9]

With no bugs and a smooth interface, this app is the epitome of simplicity, Cura enjoys customer loyalty and appreciation with satisfying booking services that help patients skip lines at hospitals, thus helping save time every time they make a trip to the doctors. [9]

NepMeds:

NepMeds is a web and mobile healthcare platform that combines medicine purchasing, doctor consultations, laboratory services, vaccination information, and digital health record storage. It emphasizes secure online transactions and affordability. The platform also includes a health information section containing educational articles on common diseases and wellness topics. [9]

To encourage adoption, NepMeds offers referral rewards and regularly updates its health feed. Planned enhancements such as medicine reminders and health monitoring tools indicate its intention to expand service offerings. [9]

Drugs Nepal:

Drugs Nepal is a brilliant simple single-purpose app that offers free, detailed information about thousands of drugs available in Nepal. Its main advantage is that it is available for offline use after the first time of loading the app, by storing all the information locally. The user needs to simply open up the app when connected to WiFi or mobile data and the updated information gets automatically synced for offline use again. [9]

While the application is useful as a quick reference tool, inconsistencies in data presentation and incomplete information limit its reliability. Improvements in data quality could enhance its practical value. [9]

Hamro Swasthya:

An official COVID-19 tracking app launched in 2020. It uses Bluetooth for contact tracing and provides features like case updates, self-assessment tests, and a plasma donation registry. While limited to pandemic-related functions, it demonstrates the government's move toward digital health tools. The Ministry of Health and Population officially launched this app in April 2020 as a quick response measure to the global Coronavirus pandemic. It requests enabling of the Bluetooth feature to help improve contact tracing and track people who might accidentally spread Covid-19. It also offers a warning feature when the user is in close proximity to a device (individual), who is in the red zone. [9]

2.2 LITERATURE REVIEW

Different researchers and developers have proposed various approaches and frameworks for improving hospital management and digital healthcare systems. Some of the notable works and existing systems relevant to this study are discussed below.

S. Shah et al. examined Nepal's developing digital health landscape and discussed platforms such as Hamro Doctor and NepMeds, which have expanded access to teleconsultation, online pharmacy, and remote health services. While these solutions improve communication and accessibility, the study noted that they are largely service-focused and do not provide comprehensive hospital management functions like inpatient records, bed allocation, or centralized administration. [9]

E. O. Boadu and G. K. Armah highlighted the importance of Role-Based Access Control (RBAC) in hospital systems. Their work demonstrated that RBAC restricts data access to authorized users, offering a scalable and auditable security framework. SwasthyaTrack adopts this approach by defining roles such as patient, doctor, nurse, and administrator to maintain data confidentiality and accountability. [4]

B. N. Bartlett et al. proposed a framework for improving inpatient bed management and patient flow, particularly in resource-constrained hospitals. Their findings showed that real-time bed tracking reduces congestion and enhances utilization. This supports SwasthyaTrack's bed reservation module, which streamlines bed availability monitoring and assignment. [3]

IBM Healthcare presented an in-depth report on healthcare interoperability, discussing standards such as HL7 FHIR for the seamless exchange of medical data across systems.

The report highlighted that lack of interoperability often leads to fragmented patient care and data duplication. SwasthyaTrack addresses this limitation by designing its database structure to support modular expansion and integration with future interoperable services such as electronic lab systems and national health repositories. [5]

Guidelines from the U.S. Department of Health & Human Services (HHS) and the Centers for Disease Control and Prevention (CDC) under Health Insurance Portability and Accountability (HIPAA) outline essential safeguards for protecting electronic health information. Drawing from these principles, SwasthyaTrack implements encryption, secure authentication, password hashing, and activity logging to protect patient data. [6][8]

S. Awad et al. emphasized the importance of usability and human-centred design in healthcare information systems. Their research demonstrated that a poorly designed interface could lead to workflow inefficiencies and medical errors, whereas systems developed with clinician input improve adoption and safety. SwasthyaTrack incorporates these findings by offering distinct, role-specific interfaces for doctors, nurses, and patients to simplify operations and minimize user errors. [8]

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 SYSTEM ANALYSIS

3.1.1 Requirement Analysis

The requirement includes both the functional and non-functional requirements:

3.1.1.1 Functional Requirement

- **User Authentication:** All users must log in using a valid username and password. Access to features is role-based.
- **Patient Registration:** Patients can register by providing their personal and contact details.
- **Appointment Management:** Patients can book, modify, or cancel appointments with available doctors. Doctors and nurses can view and manage appointment schedules.
- **Bed Management:** The system allows staff or administrators to view, allocate, and update hospital bed availability.
- **Medical Record Management:** Doctors can write prescriptions, and nurses can upload patient reports. Patients can view and download their reports and prescriptions.
- **Admin Controls:** The admin can manage users (add, edit, or delete accounts), oversee hospital data, and reset user passwords.
- **Report Generation:** The system can generate reports on appointments, bed usage, and user activities.
- **Secure File Upload:** Patients and nurses can upload medical reports (PDF format) which are securely stored for authorized access.

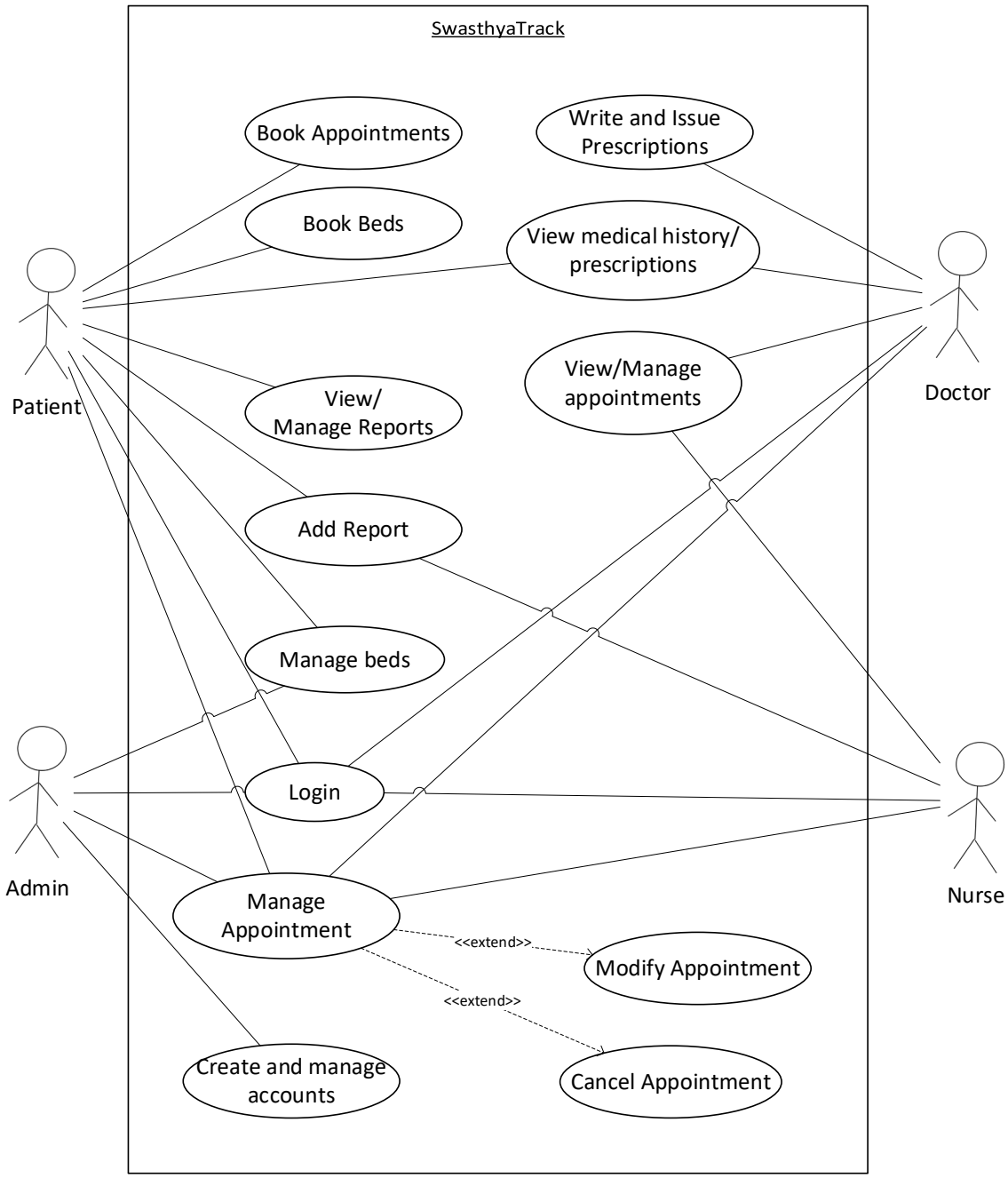


Figure 3.1 Use Case Diagram of SwasthyaTrack

3.1.1.2 Non-functional Requirement

- **Scalability:** The system architecture should be designed with scalability in mind, allowing it to handle an increasing number of users, data volume, and functional modules as the hospital expands. Future enhancements such as additional departments, advanced reporting features, or integration with external systems should be accommodated without requiring significant architectural changes or redevelopment.

- **Reliability:** The system should operate consistently and dependably with minimal downtime. It must ensure uninterrupted access to critical functionalities for hospital staff and patients, even during peak usage hours. Proper error handling, backup mechanisms, and fault tolerance should be implemented to maintain system stability and data integrity.
- **Usability:** The user interface should be intuitive, clean, and easy to navigate so that users with basic computer literacy can operate the system efficiently. Clear labels, logical workflows, and consistent design elements should be used to reduce the learning curve and minimize user errors during daily operations.
- **Compatibility:** The application should function correctly across major web browsers such as Google Chrome, Microsoft Edge, and Mozilla Firefox. It should also be compatible with different operating systems to ensure consistent behavior and appearance, providing a uniform user experience regardless of the platform used.
- **Maintainability:** The codebase should follow modular and well-documented design principles, making it easier to debug, update, and enhance over time. Proper separation of concerns and adherence to coding standards will allow developers to implement changes or fix issues efficiently without impacting other parts of the system.

3.1.2 Feasibility Analysis

Feasibility analysis evaluates whether the proposed system is practical and achievable within given resources and constraints. SwasthyaTrack is analyzed under four aspects: technical, operational, economic, and schedule feasibility.

3.1.2.1 Technical Feasibility

The project is technically feasible because it is developed using open-source technologies under the LAMP stack (Linux, Apache, MySQL, PHP). The system is hosted and tested using WSL2 (Windows Subsystem for Linux 2), providing a Linux environment within Windows. Docker is used for containerization to ensure consistent performance and ease of deployment. All tools, frameworks, and software required for development are readily available and well-documented, making implementation straightforward for the project team.

3.1.2.2 Operational Feasibility

Operational feasibility examines whether the system will function effectively in its real-world environment. SwasthyaTrack aligns well with hospital workflows and user expectations. It simplifies appointment handling, record management, and communication among hospital staff. Each user role is clearly defined, ensuring that tasks are organized and easy to perform. The system also reduces paperwork and errors, making daily hospital operations faster and more efficient.

3.1.2.3 Economic Feasibility

The project is economically feasible because it uses open-source software, which eliminates licensing costs. The system is developed for academic purposes, so no additional budget is required beyond the existing computing resources. Long-term cost savings are expected through reduced paperwork, time savings, and improved data accuracy.

3.1.2.4 Schedule Feasibility

The project was planned and executed within the given academic timeline of the semester. Each phase requirement gathering, design, development, testing, and documentation was completed within a fixed schedule. The development process followed short iterative cycles, ensuring that the project was completed on time and met the defined objectives.

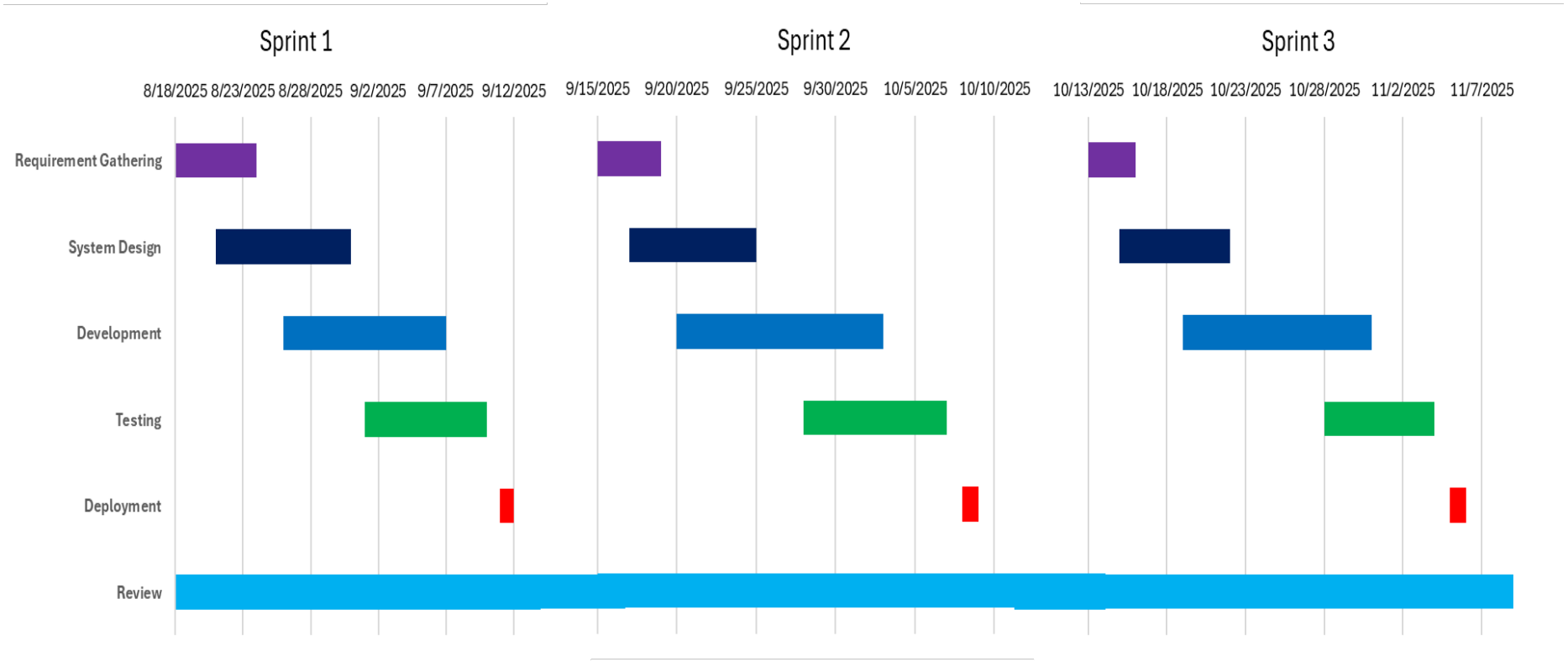


Figure 3.2 Schedule Timeline

3.1.3 Data Modelling

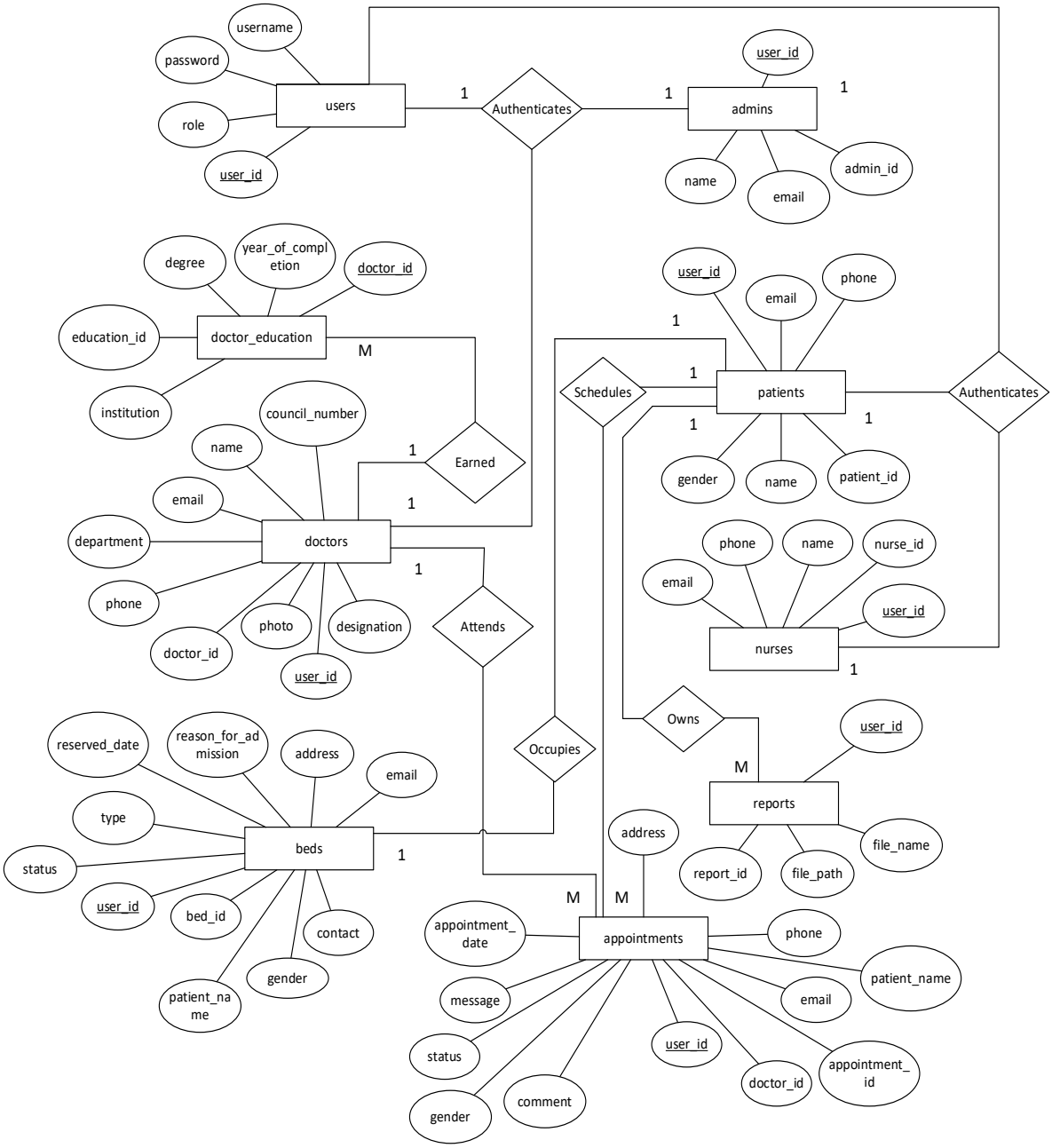


Figure 3.3 Entity Relation Diagram of SwasthyaTrack

3.1.4 Process Modeling

The DFD illustrates how data will move across SwasthyaTrack's modules. Patients, Doctors, Nurses, and Admins act as external entities that exchange information with the system. Patient can make appointment requests and then flow into processes like Appointment Management and Bed Allocation. These processes interact with central data stores including Patient Records, Appointments, Prescriptions. Doctors and Nurses update records and prescriptions, while Administrators manage accounts and hospital reports. The diagram highlights the structured flow of information, ensuring accurate data exchange and role-based access throughout the system.

Figure 3.4 DFD Level-0 of SwasthyaTrack

Figure 3.5 DFD Level-1 of SwasthyaTrack

3.2 SYSTEM DESIGN

The architectural design, database schema design, interface design and physical DFD of the SwasthyaTrack are included here.

3.2.1 Architectural Design

Architectural design involves identifying the sub-systems within a system and defining how they communicate and interact, ensuring efficient performance, security, and

scalability. For **Swasthyatrack**, the architecture follows a three-tier client–server architecture consisting of the presentation layer, application layer, and data layer. This architecture separates the user interface, business logic, and data management functions, making the system modular, scalable, and easier to maintain.

Figure 3.6 Architectural Diagram

1. **Presentation Layer (Client Side):**
   - Built using HTML, CSS, and JavaScript.
   - Handles all user interactions login, appointment booking, viewing records, etc.
   - Runs in the user's web browser and communicates with the backend using HTTP requests.
2. **Application Layer (Server Side):**
   - Implemented in PHP.
   - Processes user requests, performs authentication, manages business logic (appointments, prescriptions, bed management, etc.), and interacts with the database.
3. **Data Layer (Database Server):**
   - Managed by MySQL.
   - Stores user data, appointments, prescriptions, reports, and other hospital information.
   - Ensures data integrity and secure access.

All three layers communicate through a client server model, deployed via Apache on a Linux environment (WSL2) and containerized using Docker for consistency.

3.2.2 Database Schema Design

Figure 3.7 Database Schema of SwasthyaTrack

The database schema illustrates the main entities and their relationships within the system. It includes core tables such as Patient, Doctor, Nurse, Admin, Appointment, Bed, and Reports. Each entity is connected through primary and foreign keys to ensure data consistency. The schema shows how patients, doctors, and nurses are linked through appointments and records, allowing efficient management of hospital data in a centralized database.

3.2.3 Interface Design

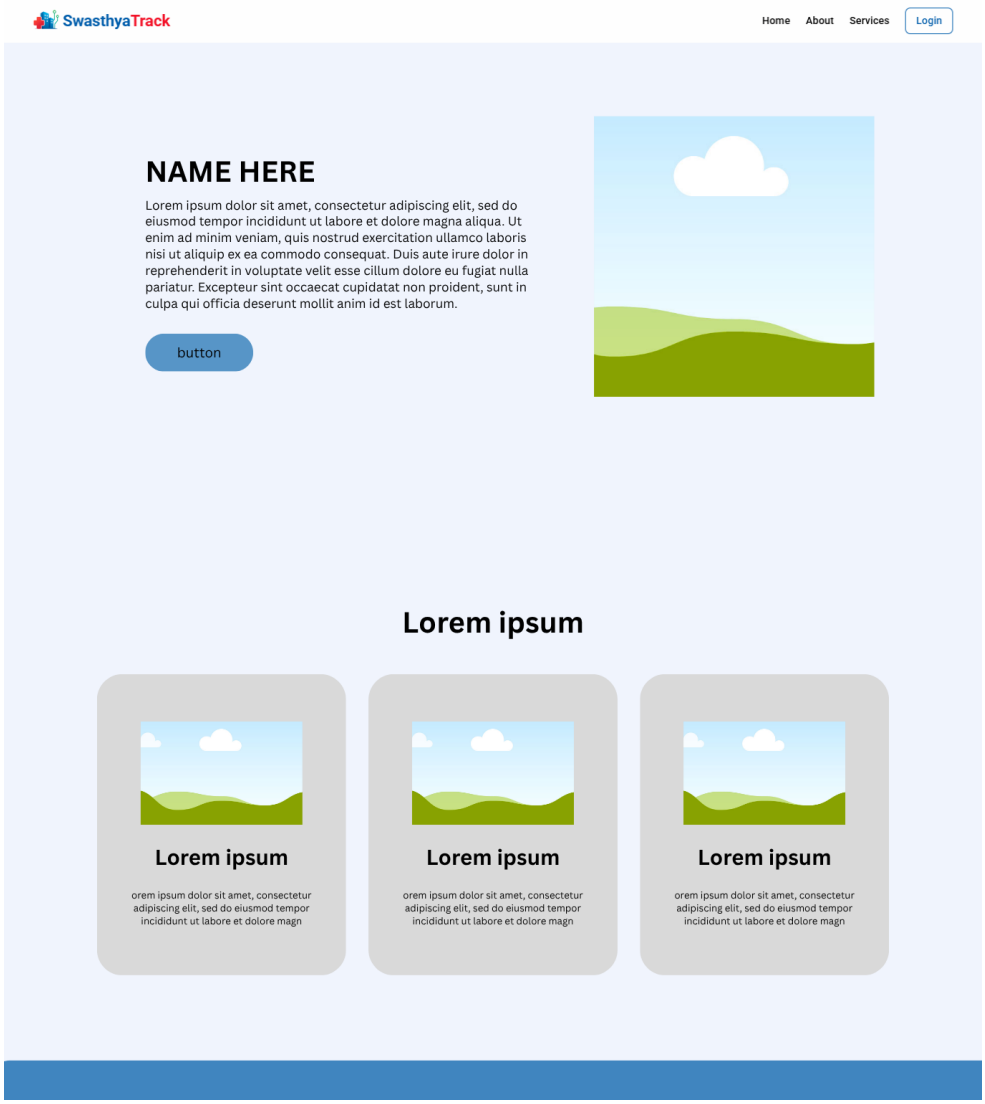


Figure 3.8 Landing Page

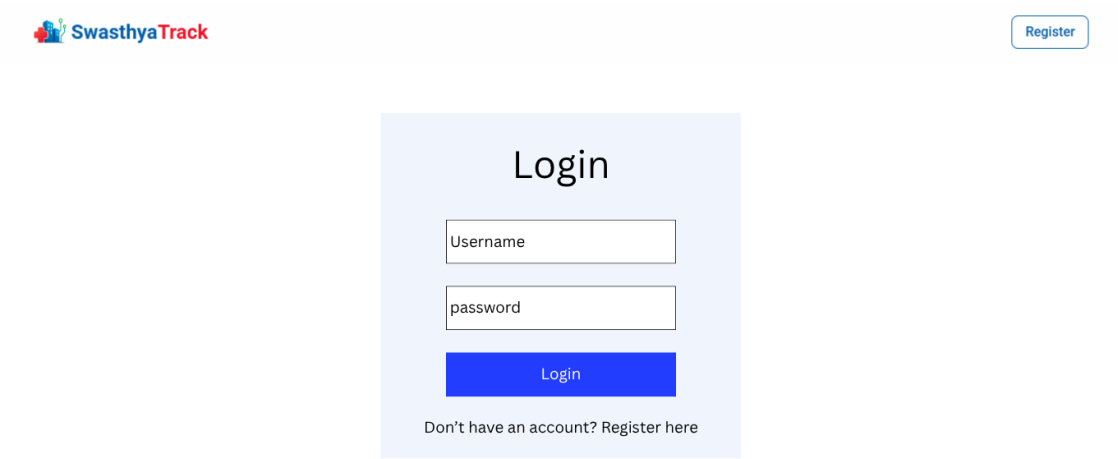


Figure 3.9 Login Page

Figure 3.10 Signup Page

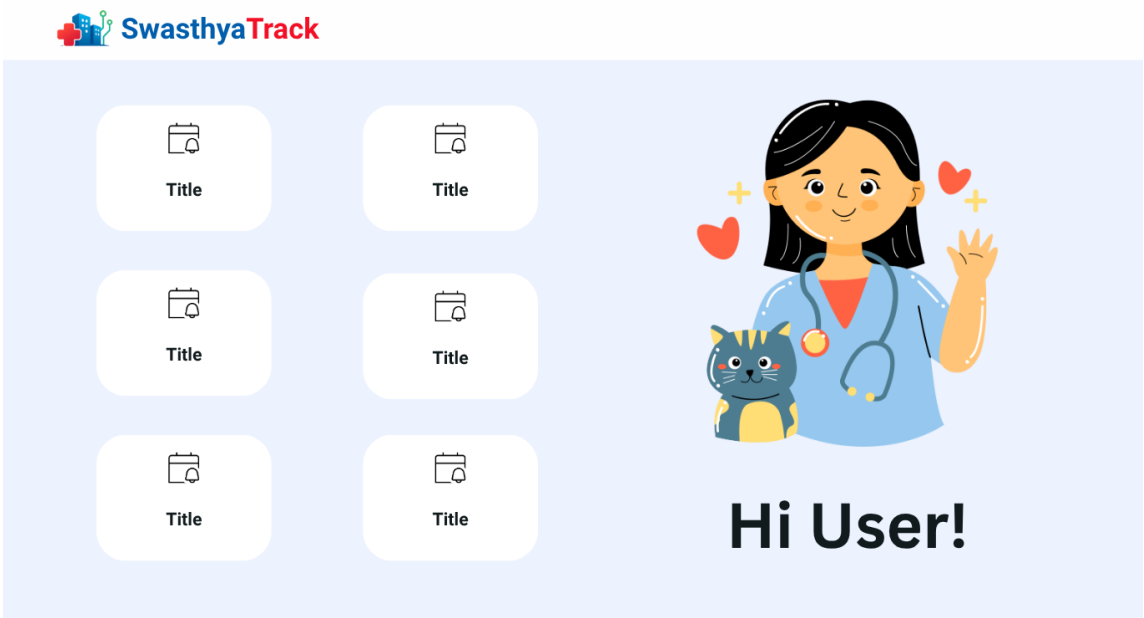


Figure 3.11 User's Dashboard

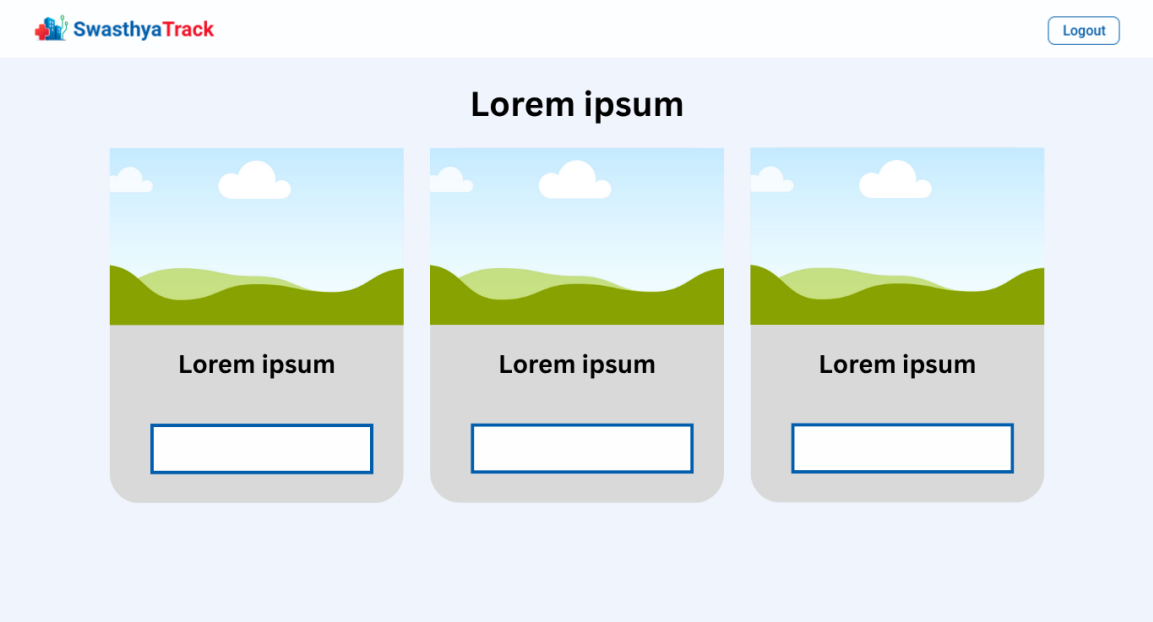


Figure 3.12 Book Bed Page

Figure 3.13 Admin & Nurse Dashboard

The interface design represents the layout and structure of the system's web pages. It focuses on simplicity, clarity, and ease of use for all users. Each user is provided with a role-based dashboard for quick access to their respective features such as appointments, reports, and system management. The design ensures smooth navigation and a consistent user experience across all modules.

3.2.4 Physical DFD

Figure 3.14 Physical DFD of SwasthyaTrack

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 IMPLEMENTATION

4.1.1 Tools Used

Table 4.1 Tools & Technology

Category	Tools/ Technology	Purpose
Operating Environment & Platform	Linux (Ubuntu via WSL2)	Provides the Linux-based environment for running Apache, PHP, MySQL, and Docker within Windows
	WSL2 (Windows Subsystem for Linux 2)	Enables execution of a Linux environment on Windows for development and testing
	Docker	Containerization platform ensuring consistent development and deployment environments
	Docker Compose	Defines and manages multi-container application services
Web & Application Technologies	Apache	Web server used to host and serve the SwasthyaTrack web application
	PHP	Server-side scripting language used to implement application logic
	MySQL	Relational database management system used to store hospital and user data
	HTML	Used to structure web pages
	CSS	Used to style and design the user interface
	JavaScript	Used for client-side interactivity and validation
Database Management	phpMyAdmin	Web-based tool for managing and monitoring the MySQL database
	dbdiagram.io	Used to design and visualize the database Schema

Development & Version Control	Visual Studio Code	Source code editor used for application development
	Git	Version control system used to track and manage code changes
	GitHub	Online repository platform used for source code hosting and collaboration
Testing & Debugging	Google Chrome	Web browser used for testing and debugging the application
Design & Documentation Tools	Draw.io	Used to create UML diagrams, DFDs, and system architecture diagrams
	Excalidraw	Used for rough sketches and initial design drafts
	Canva	Used to design report graphics, diagrams, and presentation visuals
	MS Visio	Used to create professional system diagrams and process models
Productivity & Reporting	MS Word	Used for writing project documentation and reports
	MS Excel	Used for data organization, calculations, and tabular analysis
Auxiliary Tools	Generative AI Tools	Used for conceptual clarification, troubleshooting, development guidance, and documentation support

4.1.2 Implementation Details of Modules

This section describes the internal working of each module implemented in the SwasthyaTrack system. Every module uses PHP for backend processing, MySQL for database operations, and HTML/CSS/JavaScript for user interaction. Standard procedures such as form validation, SQL prepared statements, session management, and role-based access control are applied throughout the system.

4.1.2.1 User (Patient) Modules

- **Signup Module:** In this module, the user fills a registration form that contains fields such as username, full name, email, phone number, and password. When the user clicks

on the Create button, their information is validated and a new account is created in the database. If they click on the Back button, they are redirected to the login page. Passwords are stored securely using hashing before saving them to the database.

- **Login Module:** The login module allows the user to enter the system using their username or email and password. The entered details are checked with the records stored in the database. If the credentials match, the user is logged in and redirected to their dashboard. If the details do not match, an error message is shown.
- **Book Appointment:** In this module, the user can book an appointment by selecting a doctor, a date, and an available time slot. After choosing the details, the user clicks the Book button, and the appointment is saved.
- **Manage Appointment:** The user can view all their booked appointments in this module. They can edit the appointment date or time if needed, or cancel the appointment completely. When an appointment is updated or cancelled, the changes are also updated in the database.
- **Book Bed:** This module allows users to check which beds are available in the hospital. They can choose a bed and book it for themselves. Once booked, the bed status changes to “Reserved” or “Occupied” in the system so that other users cannot book the same bed.
- **Manage Beds:** The user can view their bed reservations here. If they no longer need the bed, they can cancel the reservation. After cancellation, the bed becomes available again in the system.
- **View Reports:** The user can see the medical reports uploaded by the nurse. These reports are stored in the database with a file path, and users can download or view them as needed.
- **Add Reports:** The system allows patient uploads, the user can upload their own medical reports in this module. They select a PDF, and after clicking Upload, the file is stored on the server and linked to their account.
- **View Prescriptions:** The user can view the prescriptions written by the doctor during their appointments. The prescription includes medicines, dosage instructions, and notes. The user can also print the prescription if they need it.

- **Logout:**
  The logout button ends the user session and brings them back to the login page.

4.1.2.2 Admin Modules

- **Add Users:** In this module, the admin can add new users to the system such as doctors, nurses, other admin accounts, or staff. The admin fills a form with user details, selects the role, and clicks the Add button. The new account is then created in the database.

- **Manage Users:** The admin can view all users in the system. They can update user information, reset passwords, or delete accounts if needed. Any changes made are updated in the users table in the database.

- **Manage Appointments:** The admin sees all the appointments booked in the hospital. They can update, reassign, or cancel any appointment. This helps in cases where the user or doctor requests schedule changes.

- **Manage Beds:** The admin can add new hospital beds, update bed details, or change bed status (available, reserved, or occupied). They can also delete beds from the system if necessary.

- **Logout:** Clicking the logout button ends the admin session and redirects them to the login page.

4.1.2.3 Nurse Modules

- **Manage Appointments:** Nurses can view all appointments assigned to doctors. They help manage appointments by checking the schedule and updating the appointment status when needed.

- **Upload Reports:** Nurses can upload medical reports for patients. They select the file, enter basic details, and upload it. The system stores the file securely and links it to the patient's record.

4.1.2.4 Doctor Modules

- **Manage Appointments:** Doctors can view all their appointments for the day. They can mark an appointment as completed, cancelled, or pending based on the consultation.

- **Write Prescriptions:** In this module, doctors write prescriptions for patients by filling a simple form that includes medicine names, dosage, and instructions. After submitting, the prescription is saved in the database and becomes visible to the patient.

4.2 TESTING

Under unit testing each module of the application was manually tested and the test cases are tabulated as below:

4.2.1 Test Cases for Unit Testing

Unit testing focuses on the modules independently locate the errors. This enables the tester to detect errors in coding. It is the process of taking a module and running it in isolation from rest of the software product by using prepared test cases and comparing the actual result with the result redirected with the specifications and design of the module.

Since agile methodology was used to develop the system after each and every module was completed and was tested as a single unit.

Table 4.2 Signup Unit Test

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
UT-01	Verify successful signup with all valid input fields	{ "fullName": "Test_User", "email": "testuser@example.com", "phone": "9812345678", "username": "testuser", "gender": "Male", "password": "ValidPass123", "confirmPassword": "ValidPass123" }	User account is created successfully and success response is returned	User account created successfully	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
UT-02	Verify signup failure when full name field is empty	{ "fullName": "", "email": "testuser@example.com", "phone": "9812345678", "username": "testuser", "gender": "Male", "password": "ValidPass123", "confirmPassword": "ValidPass123" }	Input validation error indicating full name is required	Validation error returned for full name	Pass
UT-03	Verify signup failure with invalid email format	{ "fullName": "Test_User", "email": "testuser@", "phone": "9812345678", "username": "testuser", "gender": "Male", "password": "ValidPass123", "confirmPassword": "ValidPass123" }	Input validation error indicating invalid email format	Validation error returned for email format	Pass
UT-04	Verify signup failure with empty email field	{ "fullName": "Test_User", "email": "","phone": "9812345678", "username": "testuser", "gender": "Male", "password":	Input validation error indicating email is required	Validation error returned for missing email	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
		"ValidPass123","confirmPassword": "ValidPass123" }			
UT-05	Verify signup failure with invalid phone number format	{ "fullName": "Test_User", "email": "testuser@example.com", "phone": "98AB345678", "username": "testuser", "gender": "Male", "password": "ValidPass123", "confirmPassword": "ValidPass123" }	Input validation error indicating invalid phone number format	Validation error returned for phone number	Pass
UT-06	Verify signup failure when username field is empty	{ "fullName": "Test_User", "email": "testuser@example.com", "phone": "9812345678", "username": "", "gender": "Male", "password": "ValidPass123", "confirmPassword": "ValidPass123" }	Input validation error indicating username is required	Validation error returned for missing username	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
UT-07	Verify signup failure when gender is not selected	{ "fullName": "Test_User", "email": "testuser@example.com", "phone": "9812345678", "username": "testuser", "gender": "", "password": "ValidPass123","confirmPassword": "ValidPass123" }	Input validation error indicating gender selection is required	Validation error returned for gender	Pass
UT-08	Verify signup failure when password does not meet minimum length	{ "fullName": "Test_User", "email": "testuser@example.com", "phone": "9812345678", "username": "testuser", "gender": "Male", "password": "12345", "confirmPassword": "12345" }	Input validation error indicating password does not meet complexity requirements	Validation error returned for password length	Pass
UT-09	Verify signup failure when confirm password does not match password	{ "fullName": "Test_User", "email": "testuser@example.com", "phone": "9812345678", "username": "testuser", "gender": "Male", "password": "12345", "confirmPassword":	Input validation error indicating password mismatch	Validation error returned for password mismatch	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
		"ValidPass123" }			

UT-10	Verify signup failure when confirm password field is empty	{
		"fullName": "Test_User",
		"email": "testuser@example.com",
		"phone": "9812345678",
		"username": "testuser",
		"gender": "Male",
		"password": "ValidPass123","confirmPassword": ""
		}
			Input validation error indicating confirm password is required	Validation error returned for missing confirm password	Pass

UT-11	Verify signup failure with already existing username	{
		"fullName": "Test_User",
		"email": "testuser@example.com",
		"phone": "9812345678",
		"username": "testuser",
		"gender": "Male",
		"password": "ValidPass123",
		"confirmPassword": "ValidPass123"
		}
			Signup is rejected with username already exists error	Signup rejected due to duplicate username	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
UT-12	Verify error handling for missing multiple required fields	{ "fullName": "", "email": "", "phone": "", "username": "", "gender": "", "password": "", "confirmPassword": "" }	Request is rejected with required fields validation errors	Request rejected with multiple validation errors	Pass

Table 4.3 Login Unit Test

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
UT-13	Verify successful login with valid username and password	{ "username":"testuser", "password":"ValidPass123" }	User is authenticated successfully and redirected to dashboard	User authenticated successfully	Pass
UT-14	Verify login with valid username and invalid password	{ "username":"testuser", "password":"InvalidPass123" }	Authentication fails and error message is displayed	Authentication failed with error message	Pass
UT-15	Verify login with invalid username and valid password	{ "username":"wronguser", "password":"ValidPass123" }	Authentication fails and error message is displayed	Authentication failed with error message	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
UT-16	Verify login with invalid username and invalid password	{ "username":"wronguser", "password":"WrongPass123" }	Authentication fails and error message is displayed	Authentication failed with error message	Pass
UT-17	Verify login with empty username field	{ "username":"", "password":"ValidPass123" }	Input validation fails and error message is displayed	Validation error displayed for username	Pass
UT-18	Verify login with empty password field	{ "username":"testuser", "password":"" }	Input validation fails and error message is displayed	Validation error displayed for password	Pass
UT-19	Verify login with empty username and password fields	{ "username":"", "password":"" }	Input validation fails and error message is displayed	Validation error displayed for required fields	Pass

Table 4.4 Book Appointment Unit Test

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
UT-20	Verify successful appointment booking with all valid mandatory fields	{ "fullName":"Test User", "phone":"9812345678", "email":"testuser@example.com", "address":"Kathmandu", "gender":"Male", "date":"2026-01-10", "message":"Follow up consultation" }	Appointment is booked successfully and confirmation is displayed	Appointment booked successfully	Pass
UT-21	Verify appointment booking without optional message field	{ "fullName":"Test User", "phone":"9812345678", "email":"testuser@example.com", "address":"Kathmandu", "gender":"Male", "date":"2026-01-10", "message":"" }	Appointment is booked successfully without message	Appointment booked successfully	Pass
UT-22	Verify appointment booking with empty full name	{ "fullName":"", "phone":"9812345678", "email":"testuser@example.com", "address":"Kathmandu", "gender":"Male", "date":"2026-01-10",	Input validation fails and error message is displayed	Validation error displayed for full name	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
		"message":"Test" }			

UT-23	Verify appointment booking with invalid phone number format	{
		"fullName":"Test User",
		"phone":"98ABC45678",
		"email":"testuser@example.com",
		"address":"Kathmandu",
		"gender":"Male",
		"date":"2026-01-10",
		"message":"Test"
		}
			Input validation fails and error message is displayed	Validation error displayed for phone number	Pass

UT-24	Verify appointment booking with empty phone number	{
		"fullName":"Test User",
		"phone":"",
		"email":"testuser@example.com",
		"address":"Kathmandu",
		"gender":"Male",
		"date":"2026-01-10",
		"message":"Test"
		}
			Input validation fails and error message is displayed	Validation error displayed for phone number	Pass

UT-25	Verify appointment booking with invalid email format	{ "fullName":"Test User", "phone":"9812345678", "email":"testuserexample.com", "address":"Kathmandu", "gender":"Male", "date":"2026-01-10",	Input validation fails and error message is displayed	Validation error displayed for email	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
		"message":"Test" }			

UT-26	Verify appointment booking with empty email	{
		"fullName":"Test User",
		"phone":"9812345678",
		"email":"",
		"address":"Kathmandu",
		"gender":"Male",
		"date":"2026-01-10",
		"message":"Test"
		}
			Input validation fails and error message is displayed	Validation error displayed for email	Pass

UT-27	Verify appointment booking with empty address	{
		"fullName":"Test User",
		"phone":"9812345678",
		"email":"testuser@example.com",
		"address":"",
		"gender":"Male",
		"date":"2026-01-10",
		"message":"Test"
		}
			Input validation fails and error message is displayed	Validation error displayed for address	Pass

UT-28	Verify appointment booking with empty gender	{
		"fullName":"Test User",
		"phone":"9812345678",
		"email":"testuser@example.com",
		"address":"Kathmandu",
		"gender":"",
		"date":"2026-01-10",
		"message":"Test"
		}
			Input validation fails and error message is displayed	Validation error displayed for gender	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
UT-29	Verify appointment booking with past date	{ "fullName":"Test User", "phone":"9812345678", "email":"testuser@example.com", "address":"Kathmandu", "gender":"Male", "date":"2026-01-10", "message":"Test" }	Date validation fails and error message is displayed	Past dates cannot be selected	Pass
UT-30	Verify appointment booking with empty date	{ "fullName":"Test User", "phone":"9812345678", "email":"testuser@example.com", "address":"Kathmandu", "gender":"Male", "date":"", "message":"Test" }	Input validation fails and error message is displayed	Validation error displayed for date	Pass

Table 4.5 Book Bed Unit Test

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
UT-31	Verify successful bed booking for private bed with all mandatory fields	{ "bedType":"Private", "selectedBed":"#22", "patientName":"Test Patient", "gender":"Male", "contact":"9812345678", "email":"patient@example.com", "date":"2026-01-12", "address":"Kathmandu" "reasonForAdmission":"Surgery" }	Bed is booked successfully and confirmation is displayed	Bed booked successfully	Pass
UT-32	Verify successful bed booking without optional email and address	{ "bedType":"Semi-Private", "selectedBed":"#21", "patientName":"Test Patient", "gender":"Female", "contact":"9812345678", "email":"", "date":"2026-01-10", "address":"" "reasonForAdmission":"Observation" }	Bed is booked successfully and confirmation is displayed	Bed booked successfully	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
UT-33	Verify bed booking when no beds are available	–	System displays no beds available message	No beds available message displayed	Pass
UT-34	Verify bed booking with no bed selected from dropdown	{ "bedType":"Private", "selectedBed":"", "patientName":"Test Patient", "gender":"Male", "contact":"9812345678", "email":"patient@example.com", "date":"2026-01-10", "address":"Kathmandu" "reasonForAdmission":"Surgery" }	Input validation fails and error message is displayed	Validation error displayed for bed selection	Pass
UT-35	Verify bed booking with empty patient name	{ "bedType":"Private", "selectedBed":"#23", "patientName":"", "gender":"Female", "contact":"9812345678", "email":"patient@example.com", "date":"2026-01-10", "address":"Kathmandu"	Input validation fails and error message is displayed	Validation error displayed for patient name	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
		"reasonForAdmission":" Surgery" }			
UT-36	Verify bed booking with empty gender	{ "bedType":"Private", "selectedBed":"#23", "patientName":"Test Patient", "gender":"", "contact":"9812345678" , "email":"patient@example.com", "date":"2026-01-10", "address":"Kathmandu" "reasonForAdmission":" Surgery" }	Input validation fails and error message is displayed	Validation error displayed for gender	Pass
UT-37	Verify bed booking with invalid contact number format	{ "bedType":"Semi-Private", "selectedBed":"#24", "patientName":"Test Patient", "gender":"Female", "contact":"98ABC45678 ", "email":"patient@example.com", "date":"2026-01-10", "address":"Kathmandu"	Input validation fails and error message is displayed	Validation error displayed for contact number	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
		"reasonForAdmission":" Observation" }			
UT-38	Verify bed booking with empty contact number	{ "bedType":"General", "selectedBed":"#20", "patientName":"Test Patient", "gender":"Male", "contact":"", "email":"patient@example.com", "date":"2026-01-10", "address":"Kathmandu" "reasonForAdmission":" Observation" }	Input validation fails and error message is displayed	Validation error displayed for contact number	Pass
UT-39	Verify bed booking with invalid email format	{ "bedType":"Private", "selectedBed":"#23", "patientName":"Test Patient", "gender":"Male", "contact":"9812345678" , "email":"patientexample.com", "date":"2026-01-10", "address":"Katmandu" "reasonForAdmission":"	Input validation fails and error message is displayed	Validation error displayed for email	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
		Observation" }			

UT-40	Verify bed booking with empty date	{
		"bedType":"Semi-Private",
		"selectedBed":"#24",
		"patientName":"Test Patient",
		"gender":"Female",
		"contact":"9812345678",
		"email":"patient@example.com",
		"date":"",
		"address":"Kathmandu"
		"reasonForAdmission":"Observation"
		}
			Input validation fails and error message is displayed	Validation error displayed for date	Pass

UT-41	Verify bed booking with past date	{ "bedType":"General", "selectedBed":"#20", "patientName":"Test Patient", "gender":"Female", "contact":"9812345678", "email":"patient@example.com", "date":"2026-01-01", "address":"Kathmandu" "reasonForAdmission":"	Date validation fails and error message is displayed	Past dates cannot be selected	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
		Observation" }			

UT-42	Verify bed booking with empty reason for admission	{
		"bedType":"General",
		"selectedBed":"#20",
		"patientName":"Test Patient",
		"gender":"Female",
		"contact":"9812345678",
		"email":"patient@example.com",
		"date":"2026-01-10",
		"address":"Kathmandu"
		"reasonForAdmission":""
		}
			Input validation fails and error message is displayed	Validation error displayed for reason for admission	Pass

Table 4.6 Edit Appointment Unit Test

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
UT-43	Verify successful edit of appointment with all valid input fields	{ "fullName":"Test User", "phone":"9812345678", "email":"testuser@example.com", "address":"Kathmandu", 	Appointment details are updated successfully with a success response	Appointment details updated successfully	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
		"gender":"Male", "date":"2026-01-10", "message":"Follow up consultation", "status":"booked" }			
UT-44	Validate rejection when full name is empty	{ "fullName":"", "phone":"9812345678", "email":"testuser@example.com", "address":"Kathmandu", "gender":"Male", "date":"2026-01-10", "message":"Follow up consultation", "status":"booked" }	Validation error indicating full name is required	Appropriate validation error displayed	Pass
UT-45	Validate rejection for invalid phone number format	{ "fullName":"Test User", "phone":"98AB345678", "email":"testuser@example.com", "address":"Kathmandu", "gender":"Male", "date":"2026-01-10",	Validation error indicating invalid phone number format	Appropriate validation error displayed	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
		"message":"Follow up consultation", "status":"booked" }			
UT-46	Validate rejection for invalid email format	{ "fullName":"Test User", "phone":"9812345678", "email":"testuserexample.com", "address":"Kathmandu", "gender":"Male", "date":"2026-01-10", "message":"Follow up consultation", "status":"booked" }	Validation error indicating invalid email address format	Appropriate validation error displayed	Pass
UT-47	Verify successful edit when address field is empty	{ "fullName":"Test User", "phone":"9812345678", "email":"testuser@example.com", "address":"", "gender":"Male", "date":"2026-01-10", "message":"Follow up consultation",	Appointment details are updated successfully with empty address allowed	Appointment details updated successfully	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
		"status":"booked" }			

UT-48	Validate rejection when gender is not selected	{
		"fullName":"Test User",
		"phone":"9812345678",
		"email":"testuser@example.com",
		"address":"Kathmandu",
		"gender":"",
		"date":"2026-01-10",
		"message":"Follow up consultation",
		"status":"booked"
		}
			Validation error indicating invalid gender value	Gender cannot be set to empty	Pass

UT-49	Validate rejection when appointment date is in the past	{ "fullName":"Test User", "phone":"9812345678", "email":"testuser@example.com", "address":"Kathmandu", "gender":"Male", "date":"", "message":"Follow up consultation",	Validation error indicating appointment date must be a future date	Past dates cannot be selected	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
		"status":"booked" }			

UT-50	Validate rejection when appointment date is empty	{
		"fullName":"Test User",
		"phone":"9812345678",
		"email":"testuser@example.com",
		"address":"Kathmandu",
		"gender":"Male",
		"date":"",
		"message":"Follow up consultation",
		"status":"booked"
		}
			Validation error indicating appointment date is required	Appropriate validation error displayed	Pass

UT-51	Verify successful edit when appointment status is changed to cancelled	{ "fullName":"Test User", "phone":"9812345678", "email":"testuser@example.com", "address":"Kathmandu", "gender":"Male", "date":"2026-01-10", "message":"Follow up consultation",	Appointment status is updated to cancelled successfully	Appointment status updated successfully	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
		"status":"cancelled" }			

UT-52	Verify successful edit when message field is empty	{
		"fullName":"Test User",
		"phone":"9812345678",
		"email":"testuser@example.com",
		"address":"Kathmandu",
		"gender":"Male",
		"date":"2026-01-10",
		"message":"",
		"status":"booked"
		}
			Appointment details are updated successfully with empty message allowed	Appointment details updated successfully	Pass

UT-53	Validate rejection when all mandatory fields are empty	{
		"fullName":"",
		"phone":"",
		"email":"",
		"address":"",
		"gender":"Male",
		"date":"",
		"message":"",
		"status":"booked"
		}
			Validation errors are returned for all mandatory fields	All mandatory field validation errors displayed	Pass

Table 4.7 Edit Bed Reservations Unit Test

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
UT-54	Verify successful edit of bed reservation with all valid fields	{ "patientName": "Test Patient", "gender": "Male", "contact": "9812345678", "email": "testuser@example.com", "address": "Kathmandu", "reasonForAdmission": "Knee Surgery", "reservedDate": "01-10-2026" }	Bed reservation details are updated successfully	Bed reservation updated successfully	Pass
UT-55	Verify successful edit when optional email field is omitted	{ "patientName": "Test Patient", "gender": "Male", "contact": "9812345678", "email": "", "address": "Kathmandu", "reasonForAdmission": "Knee Surgery", "reservedDate": "01-10-2026" }	Bed reservation details are updated successfully without email	Bed reservation updated successfully	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
UT-56	Verify successful edit when optional address field is omitted	{ "patientName": "Test Patient", "gender": "Male", "contact": "9812345678", "email": "testuser@example.com", "address": "", "reasonForAdmission": "Knee Surgery", "reservedDate": "01-10-2026" }	Bed reservation details are updated successfully without address	Bed reservation updated successfully	Pass
UT-57	Verify edit failure when patient name is empty	{ "patientName": "", "gender": "Male", "contact": "9812345678", "email": "testuser@example.com", "address": "Kathmandu", "reasonForAdmission": "Knee Surgery", "reservedDate": "01-10-2026" }	Input validation error indicating patient name is required	Validation error returned for patient name	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
UT-58	Verify edit failure with empty gender value	{ "patientName": "Test Patient", "gender": "", "contact": "9812345678", "email": "testuser@example.com", "address": "Kathmandu", "reasonForAdmission": "Knee Surgery", "reservedDate": "01-10-2026" }	Input validation error indicating invalid gender value	Validation error returned for gender	Pass
UT-59	Verify edit failure with invalid contact number format	{ "patientName": "Test Patient", "gender": "Male", "contact": "98AB345678", "email": "testuser@example.com", "address": "Kathmandu", "reasonForAdmission": "Knee Surgery", "reservedDate": "01-10-2026" }	Input validation error indicating invalid contact number	Validation error returned for contact	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
UT-60	Verify edit failure when contact field is empty	{ "patientName": "Test Patient", "gender": "Male", "contact": "", "email": "testuser@example.com", "address": "Kathmandu", "reasonForAdmission": "Knee Surgery", "reservedDate": "01-10-2026" }	Input validation error indicating contact is required	Validation error returned for contact	Pass
UT-61	Verify edit failure with invalid email format	{ "patientName": "Test Patient", "gender": "Male", "contact": "9812345678", "email": "testuser@", "address": "Kathmandu", "reasonForAdmission": "Knee Surgery", "reservedDate": "01-10-2026" }	Input validation error indicating invalid email format	Validation error returned for email	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
UT-62	Verify edit failure when reason for admission is empty	{ "patientName": "Test Patient", "gender": "Male", "contact": "9812345678", "email": "testuser@example.com", "address": "Kathmandu", "reasonForAdmission": "", "reservedDate": "01-10-2026" }	Input validation error indicating reason for admission is required	Validation error returned for reason for admission	Pass
UT-63	Verify edit failure with invalid reserved date format	{ "patientName": "Test Patient", "gender": "Male", "contact": "9812345678", "email": "testuser@example.com", "address": "Kathmandu", "reasonForAdmission": "Knee Surgery", "reservedDate": "2026/10/01" }	Input validation error indicating invalid date format	Invalid Date format cannot be used	Pass

Table 4.8 Upload Your Medical Report Unit Test

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
UT-64	Verify successful medical report upload with valid report name and PDF file	{ "reportName": "Blood Test Report", "file": { "fileName": "blood_report.pdf", "fileType": "pdf" } }	Medical report is uploaded successfully and confirmation response is returned	Medical report uploaded successfully	Pass
UT-65	Verify upload failure when report name is empty	{ "reportName": "", "file": { "fileName": "blood_report.pdf", "fileType": "pdf" } }	Input validation error indicating report name is required	Validation error returned for report name	Pass
UT-66	Verify upload failure when PDF file is not selected	{ "reportName": "Blood Test Report", "file": { "fileName": "", "fileType": "" } }	Input validation error indicating file upload is required	Validation error returned for missing file	Pass

Test ID	Test Description	Test Input (JSON format)	Expected Result	Actual Result	Result
UT-67	Verify upload failure when uploaded file is not a PDF	{ "reportName": "Blood Test Report", "file": { "fileName": "", "fileType": "" } }	File validation error indicating only PDF files are allowed	Files except pdf cannot be selected	Pass
UT-68	Verify upload failure when both report name and file are missing	{ "reportName": "", "file": { "fileName": "", "fileType": "" } }	Request is rejected with required fields validation errors	Request rejected with validation errors	Pass

4.2.2 Test Cases for System Testing

System Testing is a level of testing that validates the complete and fully integrated software product. The purpose of a system test is to evaluate the end-to-end system specification. The system is integrated, then tested, once each module has been designed, tested, and passed all of the tests.

Table 4.9 System Testing

Test ID	Test Scenario	Precondition	Test Steps	Expected Result	Actual Result	Result
ST-01	Verify user sign-up and login flow	User is not registered	1. Sign up with valid details 2. Login with registered credentials	User account is created and login is successful	As expected,	Pass

Test ID	Test Scenario	Precondition	Test Steps	Expected Result	Actual Result	Result
ST-02	Verify login failure with invalid credentials	User account exists	1. Enter invalid username or password 2. Submit login form	Login is rejected with appropriate error message	As expected,	Pass
ST-03	Verify appointment booking with valid data	User is logged in	1. Navigate to appointment booking 2. Submit valid appointment details	Appointment is booked successfully with status set to Booked	As expected,	Pass
ST-04	Verify bed booking when selected bed category is available	User is logged in	1. Select bed category (Private/Semi-Private/General) 2. Confirm booking	Bed is booked successfully and availability is updated	As expected,	Pass
ST-05	Verify bed booking when selected bed category is unavailable	User is logged in	1. Select bed category with no availability2. Attempt booking	Booking is rejected with bed unavailable message	As expected,	Pass
ST-06	Verify appointment edit when status is Booked	Appointment status is Booked	1. Open appointment details 2. Edit appointment information	Appointment details are updated successfully	As expected,	Pass

Test ID	Test Scenario	Precondition	Test Steps	Expected Result	Actual Result	Result
ST-07	Verify appointment edit restriction when status is Completed	Appointment status is Completed	1. Attempt to edit appointment	Edit option is disabled or the appointment is not visible	As expected,	Pass
ST-08	Verify appointment edit restriction when status is Cancelled	Appointment status is Cancelled	1. Attempt to edit appointment	Edit option is disabled or blocked	As expected,	Pass
ST-09	Verify appointment cancellation	Appointment status is Booked	1. Select delete appointment option	Appointment status is updated to Cancelled	As expected,	Pass
ST-10	Verify visibility of bed reservations when status is Booked	Bed reservation status is Booked	1. Navigate to bed reservations	Reservation is visible and editable	As expected,	Pass
ST-11	Verify visibility of bed reservations when status is Occupied	Bed reservation status is Occupied	1. Navigate to bed reservations	Reservation is visible and editable	As expected,	Pass

Test ID	Test Scenario	Precondition	Test Steps	Expected Result	Actual Result	Result
ST-12	Verify bed reservation is hidden for invalid statuses	Bed reservation status is Discharged/Cancelled	1. Navigate to bed reservations	Reservation is not displayed to the user	As expected,	Pass
ST-13	Verify prescription visibility when appointment status is Completed	Appointment status is Completed	1. Navigate to prescriptions section	Prescription details are displayed successfully	As expected,	Pass
ST-14	Verify prescription access restriction when appointment is not Completed	Appointment status is Booked	1. Navigate to prescriptions section	Prescription details are not accessible	As expected,	Pass
ST-15	Verify medical report upload by user	User is logged in	1. Upload valid PDF report	Report is uploaded successfully and available to doctors	As expected,	Pass
ST-16	Verify user can view uploaded	Reports exist for user	1. Navigate to reports section	Uploaded reports are listed correctly	As expected,	Pass

Test ID	Test Scenario	Precondition	Test Steps	Expected Result	Actual Result	Result
	medical reports					
ST-17	Verify user can delete uploaded medical reports	Reports exist for user	1. Select report 2. Delete report	Report is removed successfully	As expected,	Pass
ST-18	Verify access control after logout	User is logged out	1. Attempt to access protected pages	User is redirected to login page	As expected,	Pass

CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS

5.1 LESSON LEARNT / OUTCOME

During the development of SwasthyaTrack, several important lessons were learned. The project helped in understanding how a complete web-based system is built using the LAMP stack. Working with PHP, MySQL, and Docker allowed practical experience in server-side programming, database handling, and deployment. The importance of secure coding practices, such as using prepared statements and hashing passwords, became clear during user authentication development. The project also provided experience in planning database structures, managing files, and creating role-based access controls for different users.

This project improved skills in integrating different modules like appointments, bed booking, and medical report handling into a single system. It also taught the importance of testing, debugging, and handling real-world problems such as form validation, user errors, and maintaining data consistency. Overall, the project strengthened both technical knowledge and understanding of how hospital management workflows operate in real life.

5.2 CONCLUSION

SwasthyaTrack was developed as a simple and effective hospital management system that connects patients, doctors, nurses, and administrators in one platform. The system successfully provides essential features such as user authentication, appointment booking, bed management, medical report handling, and prescription viewing. By digitizing these functions, the system helps reduce paperwork, improve communication, and save time for both hospital staff and patients.

The use of PHP and MySQL ensures a stable backend, while Docker makes the environment easier to deploy and maintain. The system also focuses on security by using hashed passwords and controlled access based on user roles. Overall, the project achieved its main objective of creating an easy-to-use hospital management system that supports basic hospital operations in a digital and organized way.

5.3 FUTURE RECOMMENDATIONS

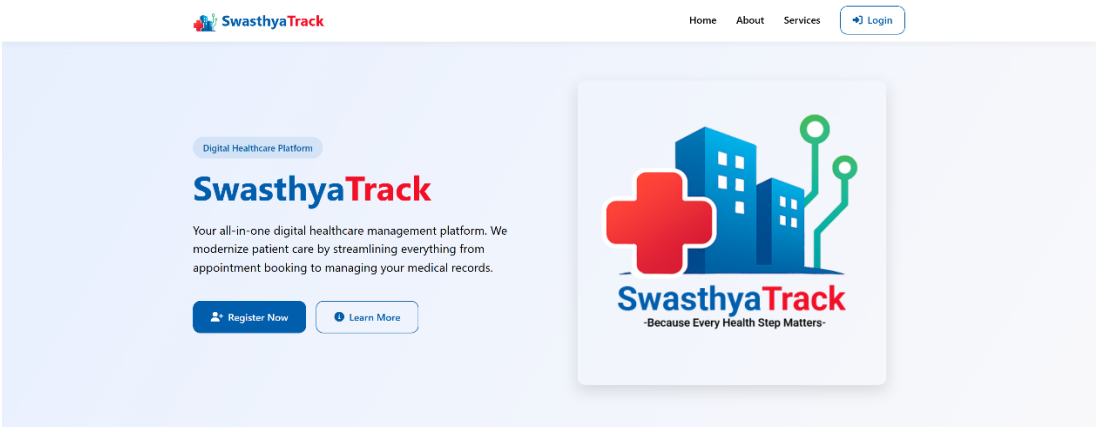
Although the system works well for basic hospital management needs, several improvements can be made in the future:

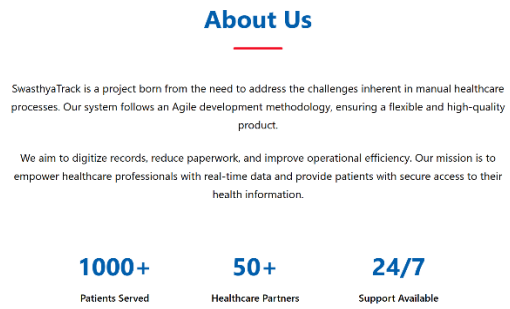
- **Add Online Payment System:** Integrate payment gateways like eSewa or Khalti to allow patients to pay hospital fees online.
- **Notification System:** Implement SMS or email notifications to remind users about upcoming appointments or bed reservations.
- **Mobile Application:** Develop a mobile app version of SwasthyaTrack for easier access and better user experience.
- **Advanced EMR Features:** Add features such as lab test history, vital sign tracking, and diagnosis summaries for more detailed medical records.
- **Analytics Dashboard:** Provide graphs, charts, and insights to help the hospital monitor patient flow, bed usage, and doctor performance.
- **Two-Factor Authentication (2FA):** Include an additional layer of security for login to protect user accounts.
- **Multi-Hospital Support:** Extend the system to manage multiple hospitals or branches under one platform.

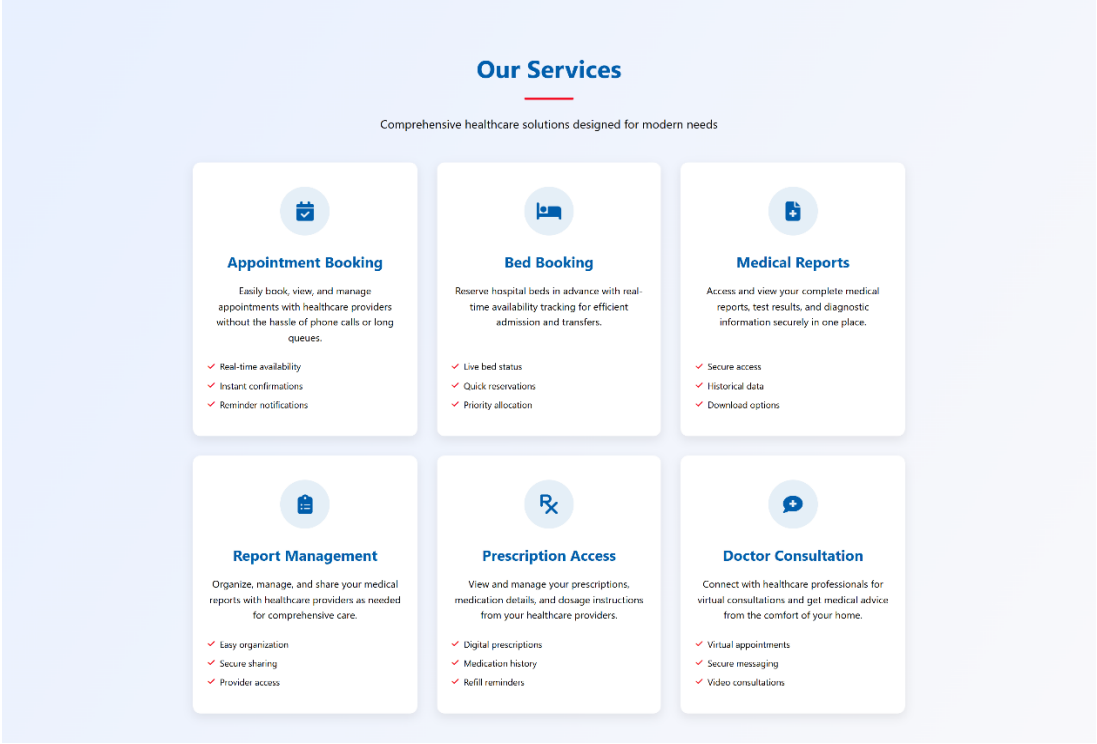
These enhancements can make SwasthyaTrack more powerful, user-friendly, and suitable for large-scale hospital operations in the future.

APPENDICES

System UI







Ready to Transform Your Healthcare Experience?

Join thousands of patients and healthcare providers using SwasthyaTrack

Get Started Today

Swasthya Track

Your trusted partner in digital healthcare management.

Quick Links

Home
About
Services

Account

Login
Register

© 2025 SwasthyaTrack. All Rights Reserved.

Swasthya Track

Home

Register

Welcome Back

Sign in to access your healthcare dashboard

Username

Password

Sign In

Patients | Doctors | Medical Staff

Don't have an account? Register here

© 2025 SwasthyaTrack. All Rights Reserved.

Swasthya Track

Home

Login

Patient Registration

Create your patient account to access healthcare services

Patient Account

Full Name

Email Address

Phone Number

Username

Select Gender

Password

Password Strength

Confirm Password

Create Patient Account

Already have an account? Login here

Healthcare professionals: Contact administrator for account creation

© 2025 SwasthyaTrack. All Rights Reserved.

SwasthyaTrack

Home Appointments Prescriptions Reports Logout

Book Appointment

Book Bed

My Appointments

Bed Reservations

My Prescriptions

My Reports

Hi Unish!

Welcome back — here's your quick access panel.

SwasthyaTrack

Home Book Appointment Appointments Reports Back

View and Select Bed Type

Private Room

- ✓ WiFi
- ✓ AC
- ✓ Attached Bathroom
- ✓ Personal TV

Select this Room

Semi-Private Room

- ✓ WiFi
- ✓ AC
- ✓ Shared Bathroom

Select this Room

General Ward Bed

- ✓ WiFi
- ✓ Shared Bathroom

Select this Room

© 2026 SwasthyaTrack. All Rights Reserved.

Meet Our Doctors

Dr. Ananya Iyer
Neurologist
Book Appointment

Dr. Anil Poudel
Senior Consultant
Book Appointment

Dr. Elena Petrova
Academic Collaborator
Book Appointment

Dr. Jun Park
Visiting Surgeon
Book Appointment

Dr. Kavita Acharya
Consultant
Book Appointment

Dr. Laxmi Neupane
Consultant
Book Appointment

Dr. Li Wei
Visiting Radiologist
Book Appointment

Dr. Meena Rai
Senior Consultant
Book Appointment

Dr. Michael Thompson
International Advisor
Book Appointment

Dr. Nisha Shrestha
Senior Consultant
Book Appointment

Dr. Prakash Joshi
Medical Officer
Book Appointment

Dr. Pratima Karki
Senior Consultant
Book Appointment

Dr. Ramesh Koirala
Consultant
Book Appointment

Dr. Sarah Johnson
Research Fellow
Book Appointment

Dr. Sarita Gurung
Consultant
Book Appointment

Dr. Suman Adhikari
Medical Officer
Book Appointment

© 2025 SwasthyaTrack. All Rights Reserved.

Book Appointment

Full Name

Phone Number

10-digit number

Email

Address

Gender

Select Gender

Pick a Date

mm/dd/yyyy

Message

Briefly describe your concern...

Confirm Booking

Dr. Priya Sharma

Senior Cardiologist

Department:

Cardiology

Designation:

Senior Cardiologist

Education:

MBBS, Tribhuvan University (2012)
Fellowship in Interventional Cardiology, Harvard Medical School (2020)

Council No:

56789

Email:

priya.sharma@hospital.com

Phone:

9812345678

© 2025 SwasthyaTrack. All Rights Reserved.

SwasthyaTrack

Home | Book Appointment | Appointments | Book Bed | Bed Reservations | Reports | Back

My Prescriptions

Search by Doctor or Patien | mm/dd/yyyy | Newest First | Apply

Doctor Name	Patient Name	Gender	Address	Message	Status	Date	Actions
Laxmi Neupane	Unish Rajak	male	Lainchaur,Kathmandu	dry cough and fever.	Completed	2026-02-09	View Prescription
Jun Park	Unish Rajak	male	Lainchaur,Kathmandu	total knee replacement surgery.	Completed	2026-02-09	View Prescription

SwasthyaTrack

Print

Patient Name: Unish Rajak
Gender: Male
Phone: 9818164355
Email: unishrajak0@gmail.com
Address: Lainchaur,Kathmandu

Doctor: Dr. Laxmi Neupane
Appointment Date: 2026-02-09
Patient Note: dry cough and fever.

Prescription

Medicine Name,Dosage,Frequency,Duration
Tab. Paracetamol (500mg),1 Tablet,Three times a day (After food),3 Days (If fever > 100◦F)
Syp. Chlorpheniramine / Dextromethorphan,10 ml,Three times a day,5 Days (For dry cough)
Tab. Pantoprazole (40 mg),1 Tablet,Once a day (Before breakfast),5 Days
Tab. Vitamin C (500mg),1 Tablet,Once a day (Chewable),10 Days

Prescribed By:
Dr. Laxmi Neupane

© SwasthyaTrack · Prescription · Printed on: 2026-02-09

Admin Panel

- Dashboard
- Add Users
- Manage Users
- Logout

Add Users

Create User (Admin)

Full Name
Email
Phone
Username
Password
Select Role
Create User

REFERENCES

[1] World Health Organization, "Gulmi Hospital launches Electronic Medical Records system," World Health Organization (WHO), 18 February 2025. [Online]. Available: https://www.who.int/nepal/news/detail/18-02-2025-gulmi-hospital-launches-electronic-medical-records-system. [Accessed August 2025].

[2] E. O. Boadu and G. K. Armah, "Role-Based Access Control (RBAC) Based Security Model for Health Information Systems: a Systematic Literature Review," BMC Health Services Research, 2025. [Online]. Available: https://bmchealthservres.biomedcentral.com/articles/10.1186/s12913-025-13293-5. [Accessed August 2025].

[3] B. N. Bartlett, L. P. Harkness and S. B. Hensley, "Optimizing inpatient bed management in a rural community-based hospital," BMC Health Services Research, 2023. [Online]. Available: https://bmchealthservres.biomedcentral.com/. [Accessed August 2025].

[4] E. O. Boadu and G. K. Armah, "Role-Based Access Control (RBAC) in Hospital Management," International Refereed Journal of Engineering and Science (IRJES), 2014. [Online]. Available: https://irjes.com/Papers/vol3-issue9/H395367.pdf#:~:text=into%20the%20RBAC%20which%20is,resources%20and%20eliminates%20security%20violations. [Accessed August 2025].

[5] IBM Healthcare, "What is Interoperability in Healthcare?," IBM THINK Blog, 2024. [Online]. Available: https://www.ibm.com/topics/interoperability-in-healthcare. [Accessed August 2025].

[6] U.S. Department of Health & Human Services (HHS), "Summary of the HIPAA Security Rule," HHS Office for Civil Rights, 2024. [Online]. Available: https://www.hhs.gov/hipaa/for-professionals/security/laws-regulations/. [Accessed August 2025].

[7] Centers for Disease Control and Prevention (CDC), "Health Insurance Portability and Accountability Act of 1996 (HIPAA)," Public Health Law Division, September 2024. [Online]. Available: https://www.cdc.gov/phlp/publications/topic/hipaa.html. [Accessed August 2025].

[8] S. Awad, R. Clay-Williams and J. Braithwaite, "Using human-centred design and human factors to support a rapid health information technology patient safety response," BMC Health Services Research, 2025. [Online]. Available: https://bmchealthservres.biomedcentral.com/. [Accessed August 2025].

[9] S. Shah, "Nepal's health-tech industry is here to stay," Nepali Times, 22 February 2021. [Online]. Available: https://nepalitimes.com/opinion/tech-away/nepal-s-health-tech-industry-is-here-to-stay. [Accessed 10 August 2025].

[10] E. O. Boadu and G. K. Armah, "Role-Based Access Control (RBAC) Based Security Model for Health Information Systems: a Systematic Literature Review," BMC Health Services Research, 2025. [Online]. Available: https://bmchealthservres.biomedcentral.com/articles/10.1186/s12913-025-13293-5. [Accessed August 2025].